

## 1) 애자일(Agile) 방법론

절차보다는 사람이 중심이 되어 변화에 유연하고 신속하게 적응하면서 효율적으로 시스템을 개발할 수 있는 신속 적응적 경량 개발 방법론으로, 개발 기간이 짧고 신속하며, 폭포수 모델에 대비되는 방법론이다.

## 2) 럼바우 데이터 모델링

## (1) 객체 모델링(Object Modeling) = 정보 모델링(Information Modeling)

시스템에서 요구하는 객체를 찾고 객체 간의 관계를 정의하여 ER 다이어그램을 만드는 과정까지의 모델링으로 객체 다이어그램을 활용하여 표현

## (2) 동적 모델링(Dynamic Modeling)

시간의 흐름에 따라 객체들 사이의 제어 흐름, 동작 순서 등의 동적인 행위를 표현하는 모델링으로 상태 다이어그램을 활용하여 표현

## (3) 기능 모델링(Functional Modeling)

프로세스들의 자료 흐름을 중심으로 처리 과정을 표현하는 모델링으로 자료 흐름도(DFD)를 활용하여 표현

## 3) 객체 지향 설계 원칙(SOLID)

## (1) 단일 책임의 원칙 (SRP : Single Responsibility Principle)

하나의 클래스는 하나의 목적을 위해서 생성되며, 클래스가 제공하는 모든 서비스는 하나의 책임을 수행하는 데 집중되어 있어야 한다는 원칙이다.

## (2) 개방 폐쇄 원칙 (OCP : Open Close Principle)

소프트웨어의 구성요소(컴포넌트, 클래스, 모듈, 함수)는 확장에는 열려 있고, 변경에는 닫혀 있어야 한다는 원칙이다.

## (3) 리스코프 치환의 원칙 (LSP : Liskov Substitution Principle)

서브 타입(상속받은 하위 클래스)은 어디서나 자신의 기반 타입(상위 클래스)으로 교체할 수 있어야 한다는 원칙이다.

## (4) 인터페이스 분리의 원칙 (ISP : Interface Segregation Principle)

한 클래스는 자신이 사용하지 않는 인터페이스는 구현하지 말아야 한다는 원칙, 객체 설계 시 특정 기능에 대한 인터페이스는 그 기능과 상관없는 부분이 변해도 영향을 받지 않아야 한다는 원칙이다.

## (5) 의존성 역전의 법칙 (DIP : Dependency Inversion Principle)

실제 사용 관계는 바뀌지 않으며, 추상을 매개로 메시지를 주고받음으로써 관계를 최대한 느슨하게 만드는 원칙이다.

## 4) 목적에 따른 디자인 패턴 유형

(1) 생성 : 객체 인스턴스 생성에 관여, 클래스 정의와 객체 생성 방식을 구조화, 캡슐화를 수행하는 패턴이다.

(2) 구조 : 더 큰 구조 형성 목적으로 클래스나 객체의 조합을 다루는 패턴이다.

(3) 행위 : 클래스나 객체들의 상호 작용하는 방법과 역할 분담을 다루는 패턴이다.

## 5) GoF 디자인 패턴의 종류

## (1) 생성 패턴 - 생빌 프로 팩팩싱

- **Builder** : 복잡한 인스턴스를 조립하여 만드는 구조로, 생성과 표기를 분리해 복잡한 객체를 생성한다.
- **Prototype** : 처음부터 일반적인 원형을 만들어 놓고, 그것을 복사한 후 필요한 부분만 수정하여 사용한다.
- **Factory Method** : 상위 클래스에서 인터페이스 정의하고 하위 클래스에서 인터페이스를 생성한다.
- **Abstract Factory** : 구체적인 클래스에 의존하지 않고 서로 연관되거나 의존적인 객체들의 조합을 만드는 인터페이스를 제공한다.  
→ 동일한 주제의 다른 팩토리를 묶음
- **Singleton** : 한 클래스에 한 객체만 존재하도록 제한한다.

## (2) 구조 패턴 - 구 브데 퍼플 프록 컴어

- **Bridge** : 기능의 클래스 계층과 구현의 클래스 계층을 연결한다.
- **Decorator** : 기존 클래스에 필요한 기능을 추가해 나가는 설계 패턴 상속의 대안으로 사용한다.
- **Facade** : 복잡한 시스템에 대하여 단순한 인터페이스를 제공함으로써 시스템 구조에 대한 파악을 쉽게 할 수 있다.
- **Flyweight** : 여러 개의 '가상 인스턴스'를 제공하여 메모리 절감 클래스의 경량화를 목적으로 한다.
- **Proxy** : '실체 객체에 대한 대리 객체', 정보 은닉, 메모리 용량 절약
- **Composite** : 객체들의 관계를 트리 구조로 구성하여 부분-전체 계층을 표현한다.
- **Adapter** : 기존에 생성된 클래스를 재사용할 수 있도록 중간에서 맞춰주는 역할을 하는 인터페이스를 만든다.

### (3) 행위 패턴 - 행 미인이 템옵 스테 비커 스트 메체

- **Mediator** : 중재자를 두고, 중재자에게 모든 걸 요구하여 통신의 빈도수를 줄여준다.
- **Interpreter** : 분리된 구문의 해석을 맡는 클래스를 각각 작성한다.
- **Iterator** : 복잡 객체의 원소를 순차적으로 접근 가능하게 해준다.
- **Template Method** : 어떤 작업을 처리하는 일부분을 서브 클래스로 캡슐화해 전체 일을 수행하는 구조는 바꾸지 않으면서 특정 단계에서 수행하는 내역을 바꾼다.
- **Observer** : 한 객체의 상태가 바뀌면 그 객체에 의존하는 다른 객체들한테 연락이 가고 자동으로 내용이 갱신되는 방법을 제공한다.
- **State** : 객체의 상태를 캡슐화하여 클래스화함으로써 그것을 참조하게 하는 방식이다.
- **Visitor** : 각 클래스 데이터 구조로부터 처리 기능을 분리하여 별도의 클래스를 만들어 놓고 해당 클래스의 메서드가 각 클래스를 돌아다니며 특정 작업을 수행하도록 만든다.
- **Command** : 재사용성이 높은 클래스를 설계하는 패턴으로 명령이 들어오면 그에 맞는 서브 클래스가 선택되어 실행된다.
- **Strategy** : 같은 알고리즘을 하나의 클래스로 캡슐화 한 뒤, 필요할 때 서로 교환해서 쓸 수 있게 한다.
- **Memento** : 객체를 이전 상태로 복구 시켜야 하는 경우, 작업취소(undo) 요청한다.
- **Chain of Responsibility** : 하드코딩을 동적으로 연결되게 처리한다.

### 6) 요구사항의 분류

(1) 기능 요구사항 : 시스템이 제공하는 기능, 서비스에 대한 요구사항이다.

(2) 비기능 요구사항 : 시스템이 수행하는 기능 이외의 사항, 시스템 구축에 대한 제약사항에 관한 요구사항이다.

#### 2단원 - 화면 설계

#### 1) UI 유형

(1) CLI (Command Line Interface) - 정적인 텍스트 기반 인터페이스 : 명령어를 텍스트로 입력하여 조작하는 사용자 인터페이스

(2) GUI (Graphical User Interface) - 그래픽 반응 기반 인터페이스

그래픽 환경을 기반으로 한 마우스나 전자펜을 이용하는 사용자 인터페이스

(3) NUI (Natural User Interface) - 직관적 사용자 반응 기반 인터페이스

키보드나 마우스 없이 신체 부위를 이용하는 사용자 인터페이스 (터치, 음성 포함)

(4) OUI (Organic User Interface) - 유기적 상호 작용 기반 인터페이스

현실에 존재하는 모든 사물이 입출력 장치로 변환할 수 있는 사용자 인터페이스

#### 2) UI 설계 원칙 - 직유학유

(1) 직관성 (Intuitiveness) : 누구나 쉽게 이해하고, 쉽게 사용할 수 있어야 함

(2) 유효성 (Efficiency) : 정확하고 완벽하게 사용자의 목표가 달성될 수 있도록 제작

(3) 학습성 (Learnability) : 초보와 숙련자 모두가 쉽게 배우고 사용할 수 있게 제작

(4) 유연성 (Flexibility) : 사용자의 요구사항을 최대한 수용하고, 실수를 방지할 수 있도록 제작

#### 3) UML 다이어그램

(1) 구조적 다이어그램(Structural Diagram) / 정적 다이어그램(Static Diagram)

- **클래스 다이어그램(Class Diagram)** : 객체 지향 모델링 시 클래스의 속성 및 연산과 클래스 간 정적인 관계를 표현한 다이어그램이다.
- **객체 다이어그램(Object Diagram)** : 클래스에 속한 사물(객체)들, 즉 인스턴스(Instance)를 특정 시점의 객체와 객체 사이의 관계로 표현한 다이어그램이다. (연관된 모든 인스턴스를 표현)
- **컴포넌트 다이어그램(Component Diagram)** : 시스템을 구성하는 컴포넌트와 그들 사이의 의존 관계를 나타내는 다이어그램이다.
- **배치 다이어그램(Deployment Diagram)** : 컴포넌트 사이의 종속성을 표현하고, 결과물, 프로세스, 컴포넌트 등 물리적 요소들의 위치를 표현하는 다이어그램이다.
- **복합체 구조 다이어그램(Composite Structure Diagram)** : 클래스나 컴포넌트가 복합 구조를 갖는 경우 그 내부 구조를 표현하는 다이어그램이다.
- **패키지 다이어그램(Package Diagram)** : 유스케이스나 클래스 등의 모델 요소들을 그룹화한 패키지들의 관계를 표현한 다이어그램이다.

## (2) 행위적 다이어그램(Behavioral Diagram) / 동적 다이어그램(Dynamic Diagram)

- 유스케이스 다이어그램(Usecase Diagram) : 시스템이 제공하고 있는 기능 및 그와 관련된 외부 요소를 사용자의 관점에서 표현하는 다이어그램이다.
- 시퀀스 다이어그램(Sequence Diagram) : 객체 간 동적 상호 작용을 시간적 개념을 중심으로 메시지 흐름으로 표현한 다이어그램이다.
- 커뮤니케이션 다이어그램(Communication Diagram) : 동작에 참여하는 객체들이 주고 받는 메시지를 표현하고, 메시지 뿐만 아니라 객체 간의 연관까지 표현하는 다이어그램이다.
- 상태 다이어그램(State Diagram) : 하나의 객체가 자신이 속한 클래스의 상태 변화 혹은 다른 객체와의 상호 작용에 따라 어떻게 변화하는지 표현하는 다이어그램이다.
- 활동 다이어그램(Activity Diagram) : 시스템이 어떤 기능을 수행하는지를 객체의 처리 로직이나 조건에 따른 처리의 흐름을 순서대로 표현하는 다이어그램이다.
- 타이밍 다이어그램(Timing Diagram) : 객체 상태 변화와 시간 제약을 명시적으로 표현하는 다이어그램이다.

## 3단원 - 데이터 입출력 구현

### 1) 데이터 모델 구성요소

- (1) 연산(Operation) : 데이터베이스에 저장된 실제 데이터를 처리하는 작업에 대한 명세
- (2) 구조(Structure) : 데이터베이스에 논리적으로 표현될 대상으로서의 개체 타입과 개체 타입들 간의 관계
- (3) 제약조건(Constraint) : 데이터베이스에 저장될 수 있는 실제 데이터의 논리적인 제약 조건

### 2) 관계 대수와 관계 해석 - 대절해비

- (1) 관계 대수(Relational Algebra) : 관계형 데이터베이스에서 원하는 정보와 그 정보를 어떻게 유도하는가를 기술하는 절차적 정형 언어
- (2) 관계 해석(Relational Calculus) : 튜플 관계 해석과 도메인 관계 해석을 하는 비절차적 언어

### 3) 관계 데이터 모델의 구성요소

- (1) 릴레이션(Relation) : 행(Row)과 열(Column)로 구성된 테이블
- (2) 튜플(Tuple) : 릴레이션의 행(Row)에 해당되는 요소
- (3) 속성(Attribute) : 릴레이션의 열(Column)에 해당되는 요소
- (4) 카디널리티(Cardinality) : 튜플(Row)의 수
- (5) 차수(Degree) : 애트리뷰트(Column)의 수
- (6) 스키마(Schema) : 데이터베이스의 구조, 제약 조건 등의 정보를 담고 있는 기본적인 구조
- (7) 인스턴스(Instance) : 정의된 스키마에 따라 생성된 테이블에 실제 저장된 데이터의 집합

### 4) 정규화(Normalization)

관계형 데이터 모델에서 데이터의 중복성을 제거하여 이상 현상을 방지하고, 데이터의 일관성과 정확성을 유지하기 위해 무손실 분해하는 과정이다.

### 5) 이상현상(Anomaly)

데이터 중복성으로 인해 릴레이션을 조작할 때 발생하는 비합리적 현상이다.

- (1) 삽입 이상 : 정보 저장 시 해당 정보의 불필요한 세부정보를 입력해야 하는 경우
- (2) 삭제 이상 : 정보 삭제 시 원치 않는 다른 정보가 같이 삭제되는 경우
- (3) 갱신 이상 : 중복 데이터 중에서 특정 부분만 수정되어 중복된 값이 모순을 일으키는 경우

### 6) 데이터베이스 정규화 단계 - 원부이 결다조

- 1정규형(1NF) : 원자값으로 구성
- 2정규형(2NF) : 부분 함수 종속(Partial Functional Dependency) 제거 → 완전 함수 종속(Full Functional Dependency) 관계
- 3정규형(3NF) : 이행 함수 종속(Transitive Functional Dependency) 제거
- 보이스-코드 정규형(BCNF) : 결정자 후보 키가 아닌 함수 종속 제거
- 4정규형(4NF) : 다치(다중 값) 종속 제거
- 5정규형(5NF) : 조인 종속 제거

## 7) 반 정규화(De-Normalization) = 비정규화, 역정규화

정규화된 엔터티, 속성, 관계에 대해 성능 향상과 개발 운영의 단순화를 위해 중복, 통합, 분리 등을 수행하는 데이터 모델링 기법이다.

## 8) 키의 종류

- (1) 기본 키(Primary Key) : 테이블의 각 튜플들을 고유하게 식별하는 컬럼
- (2) 대체 키(Alternate Key) : 후보 키 중에서 기본 키로 선택되지 않은 키
- (3) 후보 키(Candidate Key) : 테이블에서 각 튜플들을 구별하는데 기준이 되는 컬럼 (기본 키와 대체키를 합친 키)
- (4) 슈퍼 키(Super Key) : 릴레이션을 구성하는 모든 튜플에 대해 유일성은 만족하지만, 최소성은 만족하지 못하는 키
- (5) 외래 키(Foreign Key) : 테이블 간의 참조 데이터 무결성을 위한 제약 조건, 한 릴레이션의 컬럼이 다른 릴레이션의 기본 키로 이용되는 키

## 9) 데이터 마이닝

대규모로 저장된 데이터 안에서 체계적이고 자동적으로 통계적 규칙이나 패턴을 찾아내는 기술이다.

## 10) 데이터베이스 정의

- (1) 통합된 데이터(Integrated Data) : 자료의 중복을 배제한 데이터의 모임
- (2) 저장된 데이터(Stored Data) : 저장 매체에 저장된 데이터
- (3) 운영 데이터(Operational Data) : 조직의 업무를 수행하는 데 필요한 데이터
- (4) 공용 데이터(Shared Data) : 여러 애플리케이션, 시스템들이 공동으로 사용하는 데이터

## 4단원 - 통합구현

### 1) EAI(Enterprise Application Integration)

기업에서 운영되는 서로 다른 플랫폼 및 애플리케이션 간의 정보를 전달, 연계, 통합이 가능하도록 해주는 솔루션이다.

#### (1) EAI 구축 유형 - 포터 메타

- 포인트 투 포인트(Point-to-point) : 가장 기초적인 애플리케이션 통합방법으로 1:1 단순 통합방법
- 허브 앤 스포크(Hub & Spoke) : 단일한 접점의 허브 시스템을 통하여 데이터를 전송하는 중앙 집중식 방법
- 메시지 버스(Message Bus) : 애플리케이션 사이 미들웨어(버스)를 두어 연계하는 미들웨어 통합 방식
- 하이브리드(Hybrid) : 그룹 내는 허브 앤 스포크 방식을 사용하고, 그룹 간에는 메시지 버스 방식을 사용하는 통합 방식

### 2) ESB(Enterprise Service Bus)

기업에서 운영되는 서로 다른 플랫폼 및 애플리케이션들 간의 하나의 시스템으로 관리 운영할 수 있도록 서비스 중심의 통합을 지향하는 아키텍처이다.

### 3) 웹 서비스 관련 용어

- (1) HTTP(HyperText Transfer Protocol) : 인터넷에서 요청과 응답에 의해 처리하는 프로토콜로 GET, POST, PUT 등의 방식을 사용한다.
- (2) 하이퍼텍스트(Hypertext) : 문장이나 단어 등이 링크를 통해 서로 연결된 네트워크처럼 구성된 문서로 이미지 등을 누르면 다른 사이트로 옮겨갈 수 있도록 하이퍼링크(Hyperlink)가 걸려 있다.
- (3) HTML(HyperText Markup Language) : 운영체제에 상관없이 브라우저에서 실행되는 웹 문서를 표현하는 표준화된 마크업 언어로 웹 콘텐츠의 의미와 구조를 정의할 때 사용한다.

### 4) 웹 서비스 유형

- (1) SOAP(Simple Object Access Protocol) : HTTP, HTTPS, SMTP 등을 사용하여 XML 기반의 메시지를 네트워크 상태에서 교환하는 프로토콜이다.
- (2) WSDL(Web Service Description Language) : 웹 서비스 방식의 연계 모듈 구현 시 사용되는 언어로 웹 서비스명, 제공 위치, 메시지 포맷, 프로토콜 정보 등 웹 서비스에 대한 상세 정보가 기술된 XML 형식으로 구현되어 있는 언어이다.
- (3) UDDI(Universal Description, Discovery and Integration) : 웹 서비스에 대한 정보인 WSDL을 등록하고 검색하기 위한 저장소로 공개적으로 접근, 검색이 가능한 레지스트리이자 표준이다.

### 5) IPC(Inter-Process Communication) - 주요 기법 : 메시지 큐, 공유 메모리, 소켓, 세마포어

운영체제에서 프로세스 간 서로 데이터를 주고받기 위한 통신 기술이다.

## 5단원 - 인터페이스 구현

### 1) 인터페이스 기능 구현

- (1) JSON(Javascript Object Notation) : 속성-값(Attribute-Value Pair) 쌍 또는 “키-값 쌍”으로 이루어진 데이터 오브젝트를 전달하기 위해 인간이 읽을 수 있는 텍스트를 사용하는 개방형 표준 포맷이다.
- (2) XML(Extensible Markup Language) : HTML의 단점을 보완한 인터넷 언어로, SGML의 복잡한 단점을 개선한 특수한 목적을 갖는 마크업 언어이다.
- (3) AJAX(Asynchronous Javascript And XML) : 자바스크립트를 사용하여 웹 서버와 클라이언트 간 비동기적으로 XML 데이터를 교환하고 조작하기 위한 웹 기술이다.
- (4) REST(Representational State Transfer) : 웹과 같은 분산 하이퍼미디어 환경에서 자원의 존재/상태 정보를 표준화된 HTTP 메서드로 주고받는 웹 아키텍처이다.

## 6단원 - 프로그래밍 언어 활용

### [C언어]

#### 1) 식별자 표기법

- (1) 카멜 표기법(Camel Case) : 식별자 표기 시에 여러 단어가 이어지면 첫 단어 시작만 소문자로 표시하고, 각 단어의 첫 글자는 대문자로 지정하는 표기법 → inputFuction
- (2) 파스칼 표기법(Pascal Case) : 식별자 표기 시에 여러 단어가 이어지면 각 단어의 첫글자는 대문자로 지정하는 표기법 → InputFunction
- (3) 스네이크 표기법(Snake Case) : 식별자 표기 시에 여러 단어가 이어지면 단어 사이에 언더바를 넣는 표기법 → input\_function
- (4) 헝가리안 표기법(Hungarian Case) : 식별자 표기 시 두어에 자료형을 붙이는 표기법 int - n, char - c, 문자열 - sz → nScore

#### 2) 시프트 연산자

- (1) << : 왼쪽 값을 오른쪽 값만큼 비트를 왼쪽으로 이동하는 연산자
- (2) >> : 왼쪽 값에 오른쪽 값만큼 부호 비트를 채우면서 오른쪽으로 이동하는 연산자  
→ <<로 1비트 이동시킬 때마다 2배씩 증가하게 되고, >>로 1비트 이동시킬 때마다 2배씩 감소하게 된다.

#### 3) 문자열 함수

- (1) strcat : 문자열끼리 연결하는 함수이다.
- (2) strcpy : 문자열을 복사하는 함수이다.
- (3) strcmp : 문자열을 비교하는 함수이다.
- (4) strlen : 문자열의 길이를 알려주는 함수이다.
- (5) strrev : 문자열을 거꾸로 뒤집는 함수이다.
- (6) strchr : 문자열 내 일치하는 문자가 있는지 검사하는 함수이다.

#### 4) 포인터(Pointer) : 변수의 주솟값을 저장하는 공간이다.

- 자료형 뒤에 \*를 붙이면 주소를 저장하는 포인터 변수라는 의미이고, 일반 변수명에 &붙이면 해당 변수명의 주솟값이다.
- 구조체는 일반 구조체 변수로 접근할 때는 . 으로 접근하고, 구조체 포인터로 접근할 때는 ->로 접근한다.

### [자바]

#### 1) 바이트 크기

short - 2byte / int - 4byte / long - 8byte

#### 2) 클래스 접근 제어자 종류

- (1) public : 외부의 모든 클래스에서 접근이 가능한 접근 제어자
- (2) protected : 같은 패키지 내부에 있는 클래스, 하위 클래스(상속받은 경우)에서 접근이 가능한 접근 제어자. 자기 자신과 상속받은 하위 클래스 둘 다 접근이 가능한 접근 제어자
- (3) default : 접근 제어자를 명시하지 않은 경우로 같은 패키지 내부에 있는 클래스에서 접근이 가능한 접근 제어자 (자바에만 존재하는 접근 제어자이다.)
- (4) private : 같은 클래스 내에서만 접근이 가능한 접근 제어자

- 클래스는 객체를 생성하기 위해 변수와 메서드를 정의하는 틀이므로 실제 변수에 들어갈 인스턴스를 new 키워드로 생성해주어야 한다.
- this는 현재 객체를 가리키는 키워드이다. 클래스 내부의 변수와 메서드를 가리킬 수 있다.
- super 키워드를 이용하여 상위 클래스의 변수나 메서드에 접근할 수 있다.
- 객체 생성 없이 메서드를 사용하기 위해서는 static이라는 키워드를 사용하여 메서드를 선언한다.

### 3) 생성자(Constructor)

해당 클래스의 객체가 생성될 때 자동으로 호출되는 특수한 종류의 메서드이다.

### 4) 소멸자(Destructor) - finalize 메서드 (파이썬에서는 \_\_del\_\_이라는 메서드 사용)

객체의 수명이 끝났을 때 객체를 제거하기 위한 목적으로 사용되는 메서드이다.

### 5) 클래스 상속

(1) 문법 : class 자식\_클래스명 extends 부모\_클래스명

참고 - 인터페이스를 상속받을 때는 class 자식\_클래스명 implements 인터페이스\_클래스명

(2) 오버로딩(Overloading) : 동일 이름의 메서드를 매개변수만 다르게 하여 여러 개 정의할 수 있는 기능이다.

(메서드 이름 동일, 매개변수 개수나 데이터 타입이 다름)

(3) 오버라이딩(Overriding) : 하위 클래스에서 상위 클래스 메서드를 재정의할 수 있는 기능이다.

(오버라이드하고자 하는 메서드가 상위 클래스에 존재하여야 함. 매개변수 개수와 타입이 같아야 된다.)

### [파이썬]

\* 참 - True, 거짓 - False

\* print 함수를 쓰면 함수가 종료된 후에 기본으로 개행(줄바꿈)된다.

### 1) 리스트와 관련된 파이썬 함수

(1) extend() : 리스트 확장, 여러 값을 한 번에 추가

(2) pop() : 마지막 또는 지정 요소를 삭제하고 그 값을 반환

(3) reverse() : 역순으로 뒤집음

(4) append() : 리스트 마지막 요소 뒤에 값을 추가

(5) insert() : 리스트의 인덱스 위치에 값을 삽입

(6) remove() : 리스트에서 해당하는 값을 제거

## 7단원 - SQL 응용

### 1) 트랜잭션(Transaction)

인가 받지 않은 사용자로부터 데이터를 보장하기 위해 DBMS가 가져야 하는 특성이자, 데이터베이스 시스템에서 하나의 논리적 기능을 정상적으로 수행하기 위한 작업의 기본 단위이다.

### 2) 트랜잭션의 특성 - ACID

(1) 원자성(Atomicity) : 트랜잭션을 구성하는 연산 전체가 모두 정상적으로 실행되거나 모두 취소되어야 하는 성질 (전체 성공 또는 실패)

(2) 일관성(Consistency) : 시스템이 가지고 있는 고정 요소는 트랜잭션 수행 전과 트랜잭션 수행 완료 후의 상태가 같아야 하는 성질

(3) 격리성 = 고립성(Isolation) : 동시에 실행되는 트랜잭션들이 서로 영향을 미치지 않아야 한다는 성질

(4) 영속성(Durability) : 성공이 완료된 트랜잭션의 결과는 영속적으로 데이터베이스에 저장되어야 하는 성질

### 3) 트랜잭션 제어언어 = TCL(Transaction Control Language)

(1) 커밋(COMMIT) - 트랜잭션 확정 : 트랜잭션을 메모리에 영구적으로 저장하는 명령어

(2) 롤백(ROLLBACK) - 트랜잭션 취소 : 트랜잭션 내역을 저장 무효화시키는 명령어

(3) 체크포인트(CHECKPOINT) - 저장 시기 설정 : ROLLBACK을 위한 시점을 지정하는 명령어

#### 4) 데이터베이스 회복 기법 종류

- (1) **REDO** : 데이터베이스가 비정상적으로 종료되었을 때 디스크에 저장된 로그를 분석하여 트랜잭션의 시작과 완료에 대한 기록이 있는 트랜잭션들의 작업을 재 작업하는 기법이다. 데이터베이스 내용 자체가 손상된 경우, 가장 최근의 복제본을 적재한 후 이후 일어난 변경만을 로그를 이용하여 재실행함으로써 데이터베이스를 복원하는 기법이다.
- (2) **UNDO** : 데이터베이스가 비정상적으로 종료되었을 때 디스크에 저장된 로그를 분석하여 트랜잭션의 시작은 있지만, 완료 기록이 없는 트랜잭션들이 작업한 변경 내용들을 모두 취소하는 기법이다. 데이터베이스 내용 자체는 손상되지 않았지만, 변경 중이거나 변경된 내용에 대한 신뢰성을 잃어버린 경우, 모든 변경 내용을 취소하여 복원하는 기법이다.
- (3) **로그 기반 회복 기법**
  - 지연 갱신 회복 기법(Deferred Update) : 트랜잭션이 완료되기 전까지 데이터베이스에 기록하지 않는 기법
  - 즉각 갱신 회복 기법(Immediate Update) : 트랜잭션 수행 중 갱신 결과를 바로 DB에 반영하는 기법
- (4) **체크 포인트 회복 기법(Checkpoint Recovery)** : 장애 발생 시 검사점 이후에 처리된 트랜잭션에 대해서만 장애 발생 이전의 상태로 복원시키는 회복 기법
- (5) **그림자 페이징 회복 기법(Shadow Paging Recovery)** : 데이터베이스 트랜잭션 수행 시 복제본을 생성하여 데이터베이스 장애 시 이를 이용해 복구하는 기법

#### 5) 인덱스의 개념 : 검색 연산의 최적화를 위해 데이터베이스 내 값에 대한 주소 정보로 구성된 데이터 구조이다.

[데이터베이스 파일 구조]

- (1) **순차 방법** : 레코드들의 물리적 순서가 레코드들의 논리적 순서와 같게 순차적으로 저장하는 방법
- (2) **인덱스 방법** : 인덱스가 가리키는 주소를 따라 원하는 레코드에 접근할 수 있도록 하는 방법 <키 값, 주소>의 쌍으로 구성)
- (3) **해시 방법** : 키 값을 해시 함수에 대입시켜 계산한 결과를 주소로 사용하여 레코드에 접근할 수 있게 하는 방법

#### 6) 데이터 정의어 DDL(Data Definition Language)

데이터를 정의하는 언어로서 '데이터를 담는 그릇을 정의하는 언어'이다.

##### (1) DDL 대상

- 도메인(domain) : 하나의 속성이 가질 수 있는 원자값들의 집합, 속성의 데이터 타입과 크기, 제약조건 등의 정보
- 스키마(schema) : 데이터베이스 구조, 제약조건 등의 정보를 담고 있는 기본적인 구조
  - (a) 외부 스키마(external schema) : 사용자나 개발자의 관점으로 필요로 하는 데이터베이스의 논리적 구조  
(사용자 뷰를 나타냄, 서브 스키마로 불림)
  - (b) 개념 스키마(conceptual schema) : 데이터베이스의 전체적인 논리적 구조로 전체적인 뷰를 나타낸다. 개체 간의 관계, 제약 조건, 접근 권한, 무결성, 보안에 대해 정의
  - (c) 내부 스키마(internal schema) : 물리적 저장 장치의 관점에서 보는 데이터베이스 구조이다. 실제로 데이터베이스에 저장될 레코드의 형식을 정의하고 저장 데이터 항목의 표현 방법, 내부 레코드의 물리적 순서 등을 표현
- 테이블(table) : 데이터 저장 공간
- 뷰(view) : 하나 이상의 물리 테이블에서 유도되는 가상의 테이블
- 인덱스(index) : 검색을 빠르게 하기 위한 데이터 구조

##### (2) DDL 명령어 - 크알드트

- create : 데이터베이스 오브젝트 생성하는 명령어
- alter : 데이터베이스 오브젝트 변경하는 명령어 (수정)
  - alter table 컬럼 추가 문법 : alter table 테이블명 add 컬럼명 데이터타입 [제약조건];
  - alter table 컬럼 수정 문법 : alter table 테이블명 modify 컬럼명 데이터타입 [제약조건];
  - alter table 컬럼 삭제 문법 : alter table 테이블명 drop column 컬럼명;
- drop : 데이터베이스 오브젝트 삭제하는 명령어 (삭제)
  - drop table 문법 : drop table 테이블명 [cascade | restrict];
    - cascade : 참조하는 테이블까지 연쇄적으로 제거하는 옵션
    - restrict : 다른 테이블이 삭제할 테이블을 참조 중이면 제거하지 않는 옵션
- truncate : 데이터베이스 오브젝트 내용 삭제하는 명령어
  - truncate table 문법 : truncate table 테이블명;

7) 데이터 조작어 DML (Data Manipulation Language) : 데이터베이스에 저장된 저장된 자료들을 입력, 수정, 삭제, 조회하는 언어이다.

(1) select : 테이블 내 칼럼에 저장된 데이터를 조회

- 조인(Join) : 두 개 이상의 테이블을 연결하여 데이터를 검색하는 방법
  - 내부 조인(Inner Join) : 공통 존재 컬럼의 값이 같은 경우를 추출하는 기법
  - 외부 조인(Outer Join)
    - (a) 왼쪽 외부 조인(Left Outer Join) : 왼쪽 테이블의 모든 데이터와 오른쪽 테이블의 동일 데이터를 추출하는 기법
    - (b) 오른쪽 외부 조인(Right Outer Join) : 오른쪽 테이블의 모든 데이터와 왼쪽 테이블의 동일 데이터를 추출하는 기법
    - (c) 완전 외부 조인(Full Outer Join) : 양쪽의 모든 데이터를 추출하는 기법
  - 교차 조인(Cross Join) : 조인 조건이 없는 모든 데이터 조합을 추출하는 기법
  - 셀프 조인(Self Join) : 자기 자신에게 별칭을 지정한 후 다시 조인하는 기법

\*집합 연산자

- UNION : 중복 행이 제거된 쿼리 결과를 반환하는 집합 연산자
- UNION ALL : 중복 행이 제거되지 않은 쿼리 결과를 반환하는 집합 연산자
- INTERSECT : 두 쿼리 결과에 공통적으로 존재하는 결과를 반환하는 집합 연산자
- MINUS : 첫 쿼리에 있고 두 번째 쿼리에는 없는 결과를 반환하는 집합 연산자

(2) insert : 테이블 내 칼럼에 데이터를 추가 → 문법 : insert into 테이블명(속성명1, ...) values (데이터1, ...);

(3) update : 테이블 내 칼럼에 저장된 데이터를 수정 → 문법 : update 테이블명 set 속성명 = 데이터 where 조건;

(4) delete : 테이블 내 칼럼에 저장된 데이터를 삭제 → 문법 : delete from 테이블명 where 조건;

## 8) 데이터 제어어 DCL(Data Control Language)

데이터베이스 관리자가 데이터 보안, 무결성 유지, 병행 제어, 회복을 위한 관리자(DBA)가 사용하는 언어이다.

(1) grant(사용 권한 부여) : 관리자가 사용자에게 데이터베이스에 대한 권한을 부여하는 명령어 → grant 권한 on 테이블 to 사용자;

(2) revoke(사용 권한 취소) : 관리자가 사용자에게 부여했던 권한을 회수하기 위한 명령어 → revoke 권한 on 테이블 from 사용자;

## 9) 다중행 연산자 : 서버 쿼리의 결과가 여러 개의 튜플을 반환하는 다중 행 서버쿼리에서 사용되는 연산자이다.

- IN : 리턴되는 값 중에서 조건에 해당하는 값이 있으면 참
- ANY : 서버쿼리에 의해 리턴되는 각각의 값과 조건을 비교하여 하나 이상을 만족하면 참
- ALL : 값을 서버쿼리에 의해 리턴되는 모든 값과 조건 값을 비교하여 모든 값을 만족해야만 참
- EXISTS : 메인 쿼리의 비교 조건이 서버쿼리의 결과 중에서 만족하는 값이 하나라도 존재하면 참

## 10) 데이터 분석 함수 종류

(1) 집계 함수 : 여러 행 또는 테이블 전체 행으로부터 하나의 결과값을 반환하는 함수

- count : 복수 행의 줄 수를 반환하는 함수 (null은 집계에 포함하지 않음)
- sum : 복수 행의 해당 컬럼 간의 합계를 계산하는 함수
- avg : 복수 행의 해당 컬럼 간의 평균을 계산하는 함수
- max : 복수 행의 해당 컬럼 중 최대값을 계산하는 함수
- min : 복수 행의 해당 컬럼 중 최소값을 계산하는 함수
- stddev : 복수 행의 해당 컬럼 간의 표준편차를 계산하는 함수
- variance : 복수 행의 해당 컬럼 간의 분산을 계산하는 함수

(2) 그룹 함수 : 소그룹 간의 소개 및 중계 등의 중간 집계 분석 데이터를 산출하는 함수

- rollup 함수 : rollup에 의해 지정된 컬럼은 소개(소그룹의 집계) 등 중간 집계 값을 산출하기 위한 그룹 함수이다.
- cube 함수 : 결합 가능한 모든 값에 대해 다차원 집계를 생성하는 그룹 함수이다.
- grouping sets 함수 : 집계 대상 컬럼들에 대한 개별 집계를 구할 수 있으며, rollup이나 cube와는 달리 컬럼 간 순서와 무관한 결과를 얻을 수 있는 그룹 함수이다.

(3) 윈도우 함수 : 데이터베이스를 사용한 온라인 분석 처리 용도로 사용하기 위해서 표준 SQL에 추가된 기능

- rank : 동일 순위의 레코드 존재 시 후순위는 넘어감 (1 2 2 2 5 6)
- dense\_rank : 동일 순위의 레코드 존재 시에도 후순위로 넘어가지 않음 (1 2 2 2 3 4 5)
- row\_number : 동일 순위의 값이 존재해도 이와 무관하게 연속 번호를 부여 (1 2 3 4 5 6 - 2위가 3개)

- 11) **절차형 SQL** : 일반적인 개발 언어처럼 SQL 언어에서도 절차 지향적인 프로그램이 가능하도록 하는 트랜잭션 언어이다.
- **프로시저(procedure)** : 일련의 쿼리들을 마치 하나의 함수처럼 실행하기 위한 쿼리의 집합
  - **사용자 정의 함수(user-defined function)** : 일련의 SQL 처리를 수행하고, 수행 결과를 단일 값으로 반환할 수 있는 절차형 SQL
  - **트리거(Trigger)** : 데이터베이스 시스템에서 삽입, 갱신, 삭제 등의 이벤트가 발생할 때마다 관련 작업이 자동으로 수행되는 절차형 SQL
- 12) **옵티마이저(Optimizer)** : SQL을 가장 빠르고 효율적으로 수행할 최적의 처리경로를 생성해주는 DBMS 내부의 핵심엔진이다.
- **규칙기반 옵티마이저(RBO)** : 통계 정보가 없는 상태에서 사전 등 등록된 규칙에 따라 질의 실행 계획을 선택하는 옵티마이저
  - **비용기반 옵티마이저(CBO)** : 통계 정보로부터 모든 접근 경로를 고려한 질의 실행 계획을 선택하는 옵티마이저

## 8단원 - 서버 프로그램 구현

### 1) 소프트웨어 형상 관리 도구별 특징

- (1) **SVN(Subversion)** : 중앙 집중형 클라이언트-서버 형식으로 하나의 서버에서 소스를 쉽고, 유용하게 관리할 수 있게 도와주는 도구로 저장소를 만들어 그곳에 소스를 저장해서 소스 중복이나 여러 문제를 해결하기 위한 도구
- (2) **CVS(Concurrent Versions System)** : 가장 오래된 형상 관리 도구 중의 하나로서 중앙 집중형 서버 저장소를 두고 클라이언트가 접속해서 버전 관리를 실행하는 형상 관리 도구로 파일 단위로 변경 사항 관리가 가능한 도구
- (3) **RCS(Revision Control System)** : CVS와 달리 소스 파일의 수정을 한 사람만으로 제한하여 다수의 사람이 파일의 수정을 동시에 할 수 없도록 파일 잠금 방식으로 형상을 관리하는 도구
- (4) **Bitkeeper** : SVN과 비슷한 중앙 통제 방식으로 대규모 프로젝트에서 빠른 속도를 내도록 개발된 형상 관리 도구
- (5) **Git** : 로컬 저장소와 원격 저장소로 분리되어 분산 저장하는 도구로 빠른 속도에 중점을 둔 분산형 버전 관리 시스템이며, 대형 프로젝트에서 효과적이고 유용한 도구
- (6) **Clear Case** : 복수 서버, 복수 클라이언트 구조이며 서버가 부족할 때 필요한 서버를 하나씩 추가하여 확장성을 기할 수 있음

### 2) 응집도(Cohesion) : 모듈의 독립성을 나타내는 정도로, 모듈 내부 구성요소 간 연관 정도이다. ➔ 우논시절 통순간-응집도는 강하게(높게)

- (1) **우연적 응집도(Coincidental Cohesion)** : 모듈 내부의 각 구성요소가 연관이 없을 경우의 응집도
- (2) **논리적 응집도(Logical Cohesion)** : 유사한 성격을 갖거나 특정 형태로 분류되는 처리 요소들이 한 모듈에서 처리되는 경우의 응집도
- (3) **시간적 응집도(Temporal Cohesion)** : 연관된 기능이라기보다는 특정 시간에 처리되어야 하는 활동들을 한 모듈에서 처리할 경우의 응집도
- (4) **절차적 응집도(Procedural Cohesion)** : 모듈이 다수의 관련 기능을 가질 때 모듈 안의 구성요소들이 그 기능을 순차적으로 수행할 경우의 응집도
- (5) **통신적 응집도(Communication Cohesion) = 교환적 응집도** : 동일한 입력과 출력을 사용하여 다른 기능을 수행하는 활동들이 모여 있을 경우의 응집도
- (6) **순차적 응집도(Sequential Cohesion)** : 모듈 내에서 한 활동으로부터 나온 출력값을 다른 활동이 사용할 경우의 응집도
- (7) **기능적 응집도(Functional Cohesion)** : 모듈 내부의 모든 기능이 단일한 목적을 위해 수행되는 경우의 응집도

### 3) 결합도(Coupling) : 모듈 내부가 아닌 외부의 모듈과의 연관도 또는 모듈 간의 상호 의존성이다. ➔ 내공외제사-결합도는 약하게

- (1) **내용 결합도(Content Coupling)** : 다른 모듈 내부에 있는 변수나 기능을 다른 모듈에서 사용하는 경우의 결합도 / 하나의 모듈이 직접적으로 다른 모듈의 내용을 참조할 때 두 모듈은 내용적으로 결합되어 있는 경우의 결합도
- (2) **공통 결합도(Common Coupling)** : 파라미터가 아닌 모듈 밖에 선언되어 있는 전역 변수를 참조하고 전역 변수를 갱신하는 식으로 상호 작용하는 경우의 결합도 / 공유되는 공통 데이터 영역을 여러 모듈이 사용할 때의 결합도
- (3) **외부 결합도(External Coupling)** : 두 개의 모듈이 외부에서 도입된 데이터 포맷, 통신 프로토콜, 또는 디바이스 인터페이스를 공유할 경우의 결합도 / 외부 모듈에서 선언한 데이터(변수)를 외부의 다른 모듈에서 참조할 때의 결합도
- (4) **제어 결합도(Control Coupling)** : 어떤 모듈이 다른 모듈의 내부 논리 조직을 제어하기 위한 목적으로 제어 신호를 이용하여 통신하는 경우의 결합도 / 하위 모듈에서 상위 모듈로 제어 신호가 이동하여 상위 모듈에게 처리 명령을 부여하는 권리 전도 현상이 발생하는 결합도
- (5) **스탬프 결합도(Stamp Coupling)** : 모듈 간의 인터페이스로 배열이나 객체, 구조 등이 전달되는 경우의 결합도 / 두 모듈이 동일한 자료 구조를 조회하는 경우의 결합도이며, 자료 구조의 어떠한 변화는 모든 모듈에 영향을 미치게 됨
- (6) **자료 결합도(Data Coupling)** : 모듈 간의 인터페이스로 전달되는 파라미터를 통해서만 모듈 간의 상호 작용이 일어나는 경우의 결합도 / 한 모듈의 내용을 변경하더라도 다른 모듈에는 영향을 미치지 않는 상태로 가장 바람직한 결합도

## 1) SW 개발 보안의 3대 요소 - 기무가

- (1) 기밀성(Confidentiality) : 인가되지 않은 개인 혹은 시스템 접근에 따른 정보 공개 및 노출을 차단하는 특성, 인가된 사용자에게 대해서만 자원 접근이 가능해야 하는 특성
- (2) 무결성(Integrity) : 정당한 방법을 따르지 않고선 데이터가 변경될 수 없으며, 데이터의 정확성 및 완전성과 고의/악의로 변경되거나 훼손 또는 파괴되지 않음을 보장하는 특성, 인가된 사용자에게 대해서만 자원 수정이 가능하며 전송 중인 정보는 수정되지 않아야 하는 특성
- (3) 가용성(Availability) : 권한을 가진 사용자나 애플리케이션이 원하는 서비스를 지속 사용할 수 있도록 보장하는 특성, 인가된 사용자에게 가지고 있는 권한 범위 내에서 언제든지 자원 접근이 가능해야 하는 특성

## 2) SW 개발 보안 용어

- (1) 자산(Assets) : 조직의 데이터 또는 조직의 소유자가 가치를 부여한 대상
- (2) 위협(Threat) : 조직이나 기업의 자산에 악영향을 끼칠 수 있는 사건이나 행위
- (3) 취약점(Vulnerability) : 위협이 발생하기 위한 사전 조건으로 시스템의 정보 보증을 낮추는 데 사용되는 약점
- (4) 위험(Risk) : 위협이 취약점을 이용하여 조직의 자산 손실 피해를 가져올 가능성

## 3) DoS(Denial of Service) 공격

시스템을 악의적으로 공격해서 해당 시스템의 자원을 부족하게 하여 원래 의도된 용도로 사용하지 못하게 하는 공격이다.

- (1) SYN 플러딩(SYN Flooding) : TCP 프로토콜의 구조적인 문제를 이용한 공격으로 서버의 동시 가용 사용자 수를 SYN 패킷만 보내 점유하여 다른 사용자가 서버를 사용 불가능하게 하는 공격이다.
- (2) UDP 플러딩(UDP Flooding) : 대량의 UDP 패킷을 만들어 임의의 포트 번호로 전송하여 응답메시지를 생성하게 하여 지속해서 자원을 고갈시키는 공격이다.
- (3) 스머프(Smurf) / 스머핑(Smurfing) : 출발지 주소를 공격 대상의 IP로 설정하여 네트워크 전체에게 ICMP Echo 패킷을 직접 브로드캐스팅하여 마비시키는 공격이다.
- (4) 죽음의 핑(PoD ; Ping of Death) : ICMP 패킷(ping)을 정상적인 크기보다 아주 크게 만들어 전송하면 다수의 IP 단편화가 발생하고, 수신 측에서는 단편화된 패킷을 처리하는 과정에서 많은 부하가 발생하거나, 재조합 버퍼의 오버플로우가 발생하여 정상적인 서비스를 하지 못하도록 하는 공격기법이다.
- (5) 랜트 어택(Lank Attack) : 출발지(Source) IP와 목적지(Destination) IP를 같은 패킷 주소로 만들어 보냄으로써 수신자가 자기 자신에게 응답을 보내게 하여 시스템의 가용성을 침해하는 공격기법이다.
- (6) 티어 드롭 : 공격자는 IP Fragment Offset 값을 서로 중첩되도록 조작하여 전송하고, 이를 수신한 시스템이 재조합하는 과정에서 오류가 발생, 시스템의 기능을 마비시키는 DoS 공격기법이다.

## 4) DDoS(Distributed DoS) 공격 : 여러 대의 공격자를 분산 배치하여 동시에 동작하게 함으로써 특정 사이트를 공격하는 기법이다.

\* DoS와 DDoS의 차이점은 DoS는 직접 공격하고, DDoS는 공격을 하도록 지시한다. DoS는 한 사람에 의해 공격을 감행하는 반면, DDoS는 수많은 감염 호스트를 통해 공격을 감행한다.

## 5) DRDoS(Distributed Reflection DoS) 공격 : 공격자는 출발지 IP를 공격대상 IP로 위조하여 다수의 반사 서버로 요청 정보를 전송, 공격 대상자는 반사 서버로부터 다량의 응답을 받아서 서비스 거부(DoS)가 되는 공격이다.

\* DRDoS는 DDoS에 비해 공격 근원지 파악이 어렵고, 공격 트래픽 생성 효율이 DDoS보다 훨씬 크다.

## 6) 세션 하이재킹(Session Hijacking) 개념 : 케빈 미트닉이 사용했던 공격 방법 중 하나로 TCP의 세션 관리 취약점을 이용한 공격기법이다.

## 7) 네트워크 공격

- (1) 스니핑(Sniffing) : 공격대상에게 직접 공격을 하지 않고 데이터만 몰래 들여다보는 수동적 공격 기법이다.
- (2) 네트워크 스캐너(Scanner), 스니퍼(Sniffer) : 네트워크 하드웨어 및 소프트웨어 구성의 취약점 파악을 위해 공격자가 취약점을 탐색하는 공격 도구이다.
- (3) IP 스푸핑(IP Spoofing) : 침입자가 인증된 컴퓨팅 시스템인 것처럼 속여서 타깃 시스템의 정보를 빼내기 위해서 본인의 패킷 헤더를 인증된 호스트의 IP 어드레스로 위조하여 타킷에 전송하는 공격 기법이다.
- (4) ARP 스푸핑(ARP Spoofing) : 공격자가 특정 호스트의 MAC 주소를 자신의 MAC 주소로 위조한 ARP Reply를 만들어 희생자에게 지속적으로 전송하여 희생자의 ARP Cache Table에 특정 호스트의 MAC 정보를 공격자의 MAC 정보로 변경, 희생자로부터 특정 호스트로 가는 패킷을 공격자가 스니핑하는 공격 기법이다.

- (5) **ICMP Redirect 공격** : 3계층에서 스니핑 시스템을 네트워크에 존재하는 또 다른 라우터라고 알림으로써 패킷의 흐름을 바꾸는 공격 기법이다. ICMP Redirect 메시지를 공격자가 원하는 형태로 만들어서 특정 목적지로 가는 패킷을 공격자가 스니핑하는 공격 기법이다.
- (6) **트로이 목마(Trojan Horses)** : 악성 루틴이 숨어 있는 프로그램으로 겉보기에는 정상적인 프로그램으로 보이지만 실행하면 악성 코드를 실행하는 프로그램이다.

## 8) 보안 관련 용어

- (1) **스피어피싱(Spear Phishing)** : 사회 공학의 한 기법으로, 특정 대상을 선정한 후 그 대상에게 일반적인 이메일로 위장한 메일을 지속적으로 발송하여, 발송 메일의 본문 링크나 첨부된 파일을 클릭하도록 유도하여 사용자의 개인정보 탈취하는 공격 기법이다.
- (2) **스미싱(Smishing)** : 스미싱은 SMS와 피싱의 합성어로 문자메시지를 이용하여 신뢰할 수 있는 사람 또는 기업이 보낸 것처럼 가장하여 개인 비밀정보를 요구하거나 휴대폰 소액 결제를 유도하는 피싱 공격(사이버 사기)이다.
- (3) **큐싱(Qshing)** : 큐알 코드와 피싱의 합성어로 스마트폰을 이용하여 금융 업무를 처리하는 사용자에게 인증 등이 필요한 것처럼 속여 QR 앱을 내려받도록 유도, 금융 정보 등을 빼내는 피싱 공격(사이버 사기)이다.
- (4) **봇넷(Botnet)** : 악성 프로그램에 감염되어 악의적인 의도로 사용될 수 있는 다수의 컴퓨터들이 네트워크로 연결된 형태이다.
- (5) **APT 공격(Advanced Persistent Threat)** : 특정 타깃을 목표로 하여 다양한 수단을 통한 지속적이고 지능적인 맞춤형 공격 기법이다.
- (6) **공급망 공격(Supply Chain Attack)** : 소프트웨어 개발사의 네트워크에 침투하여 소스 코드의 수정 등을 통해 악의적인 코드를 삽입하거나 배포 서버에 접근하여 악의적인 파일로 변경하는 방식을 통해 사용자 PC에 소프트웨어를 설치 또는 업데이트 시에 자동적으로 감염되도록 하는 공격 기법이다.
- (7) **제로데이 공격(Zero Day Attack)** : 보안 취약점이 발견되어 널리 공표되기 전에 해당 취약점을 악용하여 이루어지는 보안 공격 기법이다.
- (8) **웜(Worm)** : 스스로 복제하여 네트워크 등의 연결을 통하여 전파하는 악성 소프트웨어 컴퓨터 프로그램이다.
- (9) **악성 봇(Malicious Bot)** : 스스로 실행되지 못하고, 해커의 명령에 의해 원격에서 제어 또는 실행이 가능한 프로그램 혹은 코드이다.
- (10) **사이버 킬체인(Cyber Kill Chain)** : 록히드 마틴의 사이버 킬체인은 공격형 방위시스템으로 지능적, 지속적 사이버 공격에 대해 7단계 프로세스별 공격분석 및 대응을 체계화한 APT 공격 방어 분석 모델이다.
- (11) **랜섬웨어(Ransomware)** : 악성 코드의 한 종류로 감염된 시스템의 파일들을 암호화하여 복호화할 수 없도록 하고, 피해자로 하여금 암호화된 파일을 인질처럼 잡고 몸값을 요구하는 악성 소프트웨어다.
- (12) **이블 트윈(Evil Twin) 공격** : 무선 Wifi 피싱 기법으로 공격자는 합법적인 Wifi 제공사처럼 행세하며 노트북이나 휴대 전화로 핫스팟에 연결한 무선 사용자들의 정보를 탈취하는 무선 네트워크 공격 기법이다.
- (13) **사회공학(Social Engineering)** : 사람들의 심리와 행동 양식을 교묘하게 이용해서 원하는 정보를 얻는 공격 기법이다.
- (14) **트러스트존(Trustzone)** : 프로세서 안에 독립적인 보안 구역을 따로 두어 중요한 정보를 보호하는 ARM 사에서 개발한 보안 기술로 프로세서 안에 독립적인 보안 구역을 따로 두어 중요한 정보를 보호하는 하드웨어 기반의 보안 기술이다.
- (15) **타이포스쿼팅(Typosquatting)** : 네티즌들이 사이트에 접속할 때 주소를 잘못 입력하거나 철자를 빠뜨리는 실수를 이용하기 위해 이와 유사한 유명 도메인을 미리 등록하는 일로, URL 하이재킹이라고도 부른다.

## 9) 서버 접근 통제 유형

- (1) **임의적 접근 통제(DAC : Discretionary Access Control)** : 주체나 그룹의 신분에 근거하여 객체에 대한 접근을 제한하는 방법
- (2) **강제적 접근 통제(MAC : Mandatory Access Control)** : 객체에 포함된 정보의 허용등급과 접근 정보에 대하여 주체가 갖는 접근 허가 권한에 근거하여 객체에 대한 접근을 제한하는 방법이다.
- (3) **역할 기반 접근 통제(RBAC : Role Based Access Control)** : 중앙 관리자가 사용자와 시스템의 상호관계를 통제하며 조직 내 맡은 역할에 기초하여 자원에 대한 접근을 제한하는 방법이다.

## 10) 3A

유무선 이동 및 인터넷 환경에서 가입자에 대한 안전하고, 신뢰성 있는 인증, 권한 검증, 과금 기능을 체계적으로 제공하는 정보 보호 기술

- (1) **인증(Authentication)** : 접근을 시도하는 가입자 또는 단말에 대한 식별 및 신분을 검증
- (2) **권한 부여(Authorization)** : 검증된 가입자나 단말에게 어떤 수준의 권한과 서비스를 허용
- (3) **계정 관리(Accounting)** : 리소스 사용에 대한 정보를 수집하고 관리하는 서비스

## 11) 인증 관련 기술

- (1) SSO(Single Sign On) : 커버로스에서 사용되는 기술로 한 번의 인증 과정으로 여러 컴퓨터상의 자원을 이용할 수 있도록 해주는 인증 기술이다.
- (2) 커버로스(Kerberos) : 1980년대 중반 MIT의 Athena 프로젝트의 일환으로 개발되었으며 클라이언트/서버 모델에서 동작하고 대칭 키 암호기법에 바탕을 둔 티켓 기반의 프로토콜이다.

## 12) 대칭 키 암호 방식

암호화와 복호화에 같은 암호 키를 쓰는 알고리즘으로 비밀키 전달을 위한 키 교환이 필요하고, 암호화 및 복호화의 속도가 빠르다.

- (1) DES(Data Encryption Standard) : 1975년 IBM에서 개발하고 미국의 연방 표준국(NIST)에서 발표한 대칭 키 기반의 블록 암호화 알고리즘이다. (블록 크기 64 bit)
- (2) SEED : 1999년 국내 한국인터넷진흥원(KISA)이 개발한 블록 암호화 알고리즘이다. (블록 크기 128bit)
- (3) AES(Advanced Encryption Standard) : 2001년 미국 표준 기술 연구소(NIST)에서 발표한 블록 암호화 알고리즘이다. (블록 크기 128bit)
- (4) ARIA(Academy Research Institute, Agency) : 2004년 국가정보원과 산학연구협회가 개발한 블록 암호화 알고리즘이다. (블록 크기 128bit)
- (5) IDEA(International Data Encryption Algorithm) : DES를 대체하기 위해 스위스 연방기술 기관에서 개발한 블록 암호화 알고리즘이다. (128bit의 키를 사용하여 64bit의 평문을 8라운드에 거쳐 64bit의 암호문을 만듦)
- (6) LFSR(Linear Feedback Shift Register) : 시프트 레지스터의 일종으로, 레지스터에 입력되는 값이 이전 상태 값들을 선형 함수로 계산되는 구조로 되어 있는 스트림 암호화 알고리즘이다.
- (7) Skipjack : 미 국가안보국(NSA)에서 개발한 Clipper 칩에 내장된 알고리즘으로 소프트웨어로 구현되는 것을 막고자 Fortezza Card에 칩 형태로 구현했다.

## 13) 비대칭 키 암호 방식(=공개키 암호 방식)

사전에 개인 키를 나눠 가지지 않은 사용자들이 안전하게 통신하는 방식이다.

- (1) 디피-헬만(Diffie-Hellman) : 최초의 공개키 알고리즘으로 W.Diffie와 M.Hellman이 1976년에 고안한 알고리즘으로서 유한 필드 내에서 이산대수의 계산이 어려운 문제를 기본 원리로 하고 있다.
- (2) RSA(Rivest shamir Adleman) : 1977년 3명의 MIT 수학 교수가 고안한 큰 인수의 곱을 소인수 분해하는 수학적 알고리즘 이용하는 공개키 암호화 알고리즘이다.
- (3) ECC(Elliptic Curve Cryptography) : 1985년 코블리치와 밀러가 RSA 암호 방식에 대한 대안으로 처음 제안했다. 타원 곡선 암호(ECC)는 유한체 위에서 정의된 타원곡선 군에서의 이산대수의 문제에 기초한 공개키 암호화 알고리즘이다.
- (4) ElGamal : T.ElGamal이 1984년에 제안한 공개키 알고리즘으로 이산대수의 계산이 어려운 문제를 기본 원리로 하고 있으며, RSA와 유사하게 전자서명과 데이터 암호복호화에 함께 사용 가능하다.

## 14) IPSec(Internet Protocol Security)

IP 계층(3계층)에서 무결성과 인증을 보장하는 인증 헤더(AH)와 기밀성을 보장하는 암호화(ESP)를 이용한 IP 보안 프로토콜이다.

## 15) 입력 데이터 검증 및 표현 취약점

- (1) XSS (Cross Site Scripting) : 검증되지 않은 외부 입력 데이터가 포함된 웹페이지가 전송되는 경우, 사용자가 해당 웹페이지를 열람함으로써 웹페이지에 포함된 부적절한 스크립트가 실행되는 공격이다.
- (2) 사이트 간 요청 위조(CSRF ; Cross-Site Request Forgery) : 사용자가 자신의 의지와는 무관하게 공격자가 의도한 행위를 특정 웹사이트에 요청하게 하는 공격이다.
- (3) SQL 삽입(Injection) : 응용 프로그램의 보안 취약점을 이용해서 악의적인 SQL 구문을 삽입, 실행시켜서 데이터베이스의 접근을 통해 정보를 탈취하거나 조작 등의 행위를 하는 기법이다.

## 16) 네트워크 보안 솔루션

- (1) 방화벽(Firewall) : 기업 내부, 외부 간 트래픽을 모니터링 하여 시스템의 접근을 허용하거나 차단하는 시스템이다.
- (2) 웹 방화벽(WAF ; Web Application Firewall) : 일반적인 네트워크 방화벽과는 달리 웹 애플리케이션 보안에 특화된 보안 장비이다.
- (3) 네트워크 접근 제어(NAC ; Network Access Control) : 단말기가 내부 네트워크에 접속을 시도할 때 이를 제어하고 통제하는 기능을 제공하는 솔루션이다.

- (4) **침입 탐지 시스템(IDS ; Intrusion Detection System)** : 네트워크에서 발생하는 이벤트를 모니터링하고 비인가 사용자에게 의한 자원접근과 보안정책 위반 행위를 실시간으로 탐지하는 시스템이다.
- (5) **침입 방지 시스템(IPS ; Intrusion Prevention System)** : 네트워크에 대한 공격이나 침입을 실시간적으로 차단하고, 유해트래픽에 대한 조치를 능동적으로 처리하는 시스템이다.
- (6) **무선 침입 방지 시스템(WIPS ; Wireless Intrusion Prevention System)** : 인가되지 않은 무선 단말기의 접속을 자동으로 탐지 및 차단하고 보안에 취약한 무선 공유기를 탐지하는 시스템이다.
- (7) **통합 보안 시스템(UTM ; Unified Threat Management)** : 방화벽, 침입 탐지 시스템(IDS), 침입 방지 시스템(IPS), VPN, 안티 바이러스, 이메일 필터링 등 다양한 보안 장비의 기능을 하나의 장비로 통합하여 제공하는 시스템이다.
- (8) **가상 사설망(VPN ; Virtual Private Network)** : 인터넷과 같은 공중망에 인증, 암호화, 터널링 기술을 활용하여 마치 전용망을 사용하는 효과를 가지는 보안 솔루션이다.
- (9) **SIEM(Security Information and Event Management)** : 다양한 보안 장비와 서버, 네트워크 장비 등으로부터 보안 로그와 이벤트 정보를 수집한 후 정보 간의 연관성을 분석하여 위협 상황을 인지하고, 침해사고에 신속하게 대응하는 보안 관제 솔루션이다.
- (10) **ESM(Enterprise Security Management)** : 방화벽, 침입 탐지 시스템(IDS), UTM, 가상 사설망 등의 여러 보안 시스템으로부터 발생한 각종 이벤트 및 로그를 통합해서 관리, 분석, 대응하는 전사적 통합 보안 관리 시스템이다.

## 17) 보안 공격 관련 용어

- (1) **워터링 홀(Watering Hole)** : 공격대상이 방문할 가능성이 있는 합법적 웹사이트를 미리 감염시켜 놓고, 피해자가 방문했을 때 피해자의 컴퓨터에 악성 프로그램을 배포하는 공격 기법이다.
- (2) **파밍(Pharming)** : 인터넷 주소창에 방문하고자 하는 사이트의 URL을 입력하였을 때 가짜 사이트(fake site)로 이동시키는 공격 기법이다.
- (3) **드라이브 바이 다운로드(Drive By Download)** : 악의적인 해커가 불특정 웹 서버와 웹 페이지에 악성 스크립트를 설치하고, 불특정 사용자 접속 시 사용자 동의 없이 실행되어 의도된 서버로 연결하여 감염시키는 공격 기법이다.
- (4) **비즈니스 스캠(Business SCAM)** : 기업 이메일 계정 도용하여 무역 거래 대금 가로채는 사이버 범죄이다.
- (5) **피싱(Phishing)** : 소셜 네트워크에서 악의적인 사용자가 지인 또는 특정 유명인으로 가장하여 불특정 다수의 정보를 탈취하는 공격 기법이다.
- (6) **사이버 킬체인(Cyber Kill Chain)** : 록히드 마틴의 사이버 킬체인은 공격형 방위시스템으로 지능적, 지속적 사이버 공격에 대해 7단계 프로세스별 공격 분석 및 대응을 체계화한 APT 공격 방어 분석 모델이다.

## 10단원 - 애플리케이션 테스트 관리

### 1) 화이트박스 테스트 - 구결조 조변다 기제데루 : 각 응용 프로그램의 내부 구조와 동작을 검사하는 소프트웨어 테스트이다.

- (1) **구문 커버리지 = 문장 커버리지(Statement Coverage)**  
프로그램 내의 모든 명령문을 적어도 한 번 수행하는 커버리지이다.
- (2) **결정 커버리지 = 선택 커버리지(Decision Coverage) = 분기 커버리지(Branch Coverage)**  
각 분기의 결정 포인트 내의 전체 조건식이 적어도 한 번은 참과 거짓의 결과를 수행하는 테스트 커버리지이다.
- (3) **조건 커버리지(Condition Coverage)**  
각 분기의 결정 포인트 내의 각 개별 조건식이 적어도 한 번은 참과 거짓의 결과가 되도록 수행하는 테스트 커버리지이다.
- (4) **조건/결정 커버리지(Condition/Decision Coverage)**  
전체 조건식뿐만 아니라 개별 조건식도 참 한 번, 거짓 한 번 결과가 되도록 수행하는 테스트 커버리지이다.
- (5) **변경 조건/결정 커버리지(Modified Condition/Decision Coverage)**  
개별 조건식이 다른 개별 조건식에 영향을 받지 않고 전체 조건식에 독립적으로 영향을 주도록 함으로써 조건/결정 커버리지를 향상시킨 커버리지이다.
- (6) **다중 조건 커버리지(Multiple Condition Coverage)**  
결정 조건 내 모든 개별 조건식의 모든 가능한 조합을 100% 보장하는 커버리지이다.
- (7) **기본 경로 커버리지 = 경로 커버리지(Base Path Coverage)** : 수행 가능한 모든 경로를 테스트하는 기법이다.
- (8) **제어 흐름 커버리지(Control Flow Testing)** : 프로그램 제어 구조를 그래프 형태로 나타내어 내부 로직을 테스트하는 기법이다.
- (9) **데이터 흐름 테스트(Data Flow Testing)** : 제어 흐름 그래프에 데이터 사용현황을 추가한 그래프를 통해 테스트하는 기법이다.
- (10) **루프 테스트(Loop Testing)** : 프로그램의 반복 구조에 초점을 맞춰 실시하는 테스트 기법이다.

## 2) 블랙박스 테스트 - 동경결상 유분패원비오

프로그램 외부 사용자의 요구사항 명세를 보면서 수행하는 테스트(기능 테스트)이다.

### (1) 동등분할 테스트 = 동치 분할 테스트, 균등분할 테스트, 동치 클래스 분해 테스트(Equivalence Partitioning Testing)

입력 데이터의 영역을 유사한 도메인별로 유효값/무효값을 그룹핑하여 대푯값 테스트 케이스를 도출하여 테스트하는 기법이다.

### (2) 경계값 분석 테스트 = 한겻값 테스트(Boundary Value Analysis Testing)

등가 분할 후 경계값 부분에서 오류 발생 확률이 높기 때문에 경계값을 포함하여 테스트 케이스를 설계하여 테스트 하는 기법이다.

### (3) 결정 테이블 테스트(Decision Table Testing)

요구사항의 논리와 발생 조건을 테이블 형태로 나열하여, 조건과 행위를 모두 조합하여 테스트하는 기법이다.

### (4) 상태 전이 테스트(State Transition Testing)

테스트 대상/시스템이나 객체의 상태를 구분하고, 이벤트에 의해 어느 한 상태에서 다른 상태로 전이되는 경우의 수를 수행하는 테스트 기법이다.

### (5) 유스케이스 테스트(Use Case Testing)

시스템이 실제 사용되는 유스케이스로 모델링 되어 있을 때 프로세스 흐름을 기반으로 테스트 케이스를 명세화하여 수행하는 테스트 기법이다.

### (6) 분류 트리 테스트(Classification Tree Method Testing)

SW의 일부 또는 전체를 트리 구조로 분석 및 표현하여 테스트 케이스를 설계하여 테스트하는 기법이다.

### (7) 페어와이즈 테스트(Pairwise Testing)

테스트 데이터값들 간에 최소한 한 번씩을 조합하는 방식이며, 이는 커버해야 할 기능적 범위를 모든 조합에 비해 상대적으로 적은 양의 테스트 세트를 구성하기 위한 테스트 기법이다.

### (8) 원인-결과 그래프 테스트(Cause-Effect Graph Testing)

그래프를 활용하여 입력 데이터 간의 관계 및 출력에 미치는 영향을 분석하여 효용성이 높은 테스트 케이스를 선정하여 테스트하는 기법이다.

### (9) 비교 테스트(Comparison Testing)

여러 버전의 프로그램에 같은 입력값을 넣어서 동일한 결과 데이터가 나오는지 비교해 보는 테스트 기법이다.

### (10) 오류 추정 테스트(Error Guessing Testing)

개발자가 범할 수 있는 실수를 추정하고 이에 따른 결함이 검출되도록 테스트 케이스를 설계하여 테스트하는 기법이다.

## 3) 테스트 시각에 따른 분류

### (1) 검증(Verification) : 소프트웨어 개발 과정을 테스트한다.

→ 개발자 혹은 시험자의 시각으로 소프트웨어가 명세화된 기능을 올바르게 수행하는지 알아보는 과정이다.

### (2) 확인(Validation) : 소프트웨어 결과를 테스트한다.

→ 사용자 시각으로 올바른 소프트웨어가 개발되었는지 입증하는 과정이다.

## 4) 테스트 목적에 따른 분류 - 회안성 구회병

### (1) 회복 테스트(Recovery Testing) : 시스템에 고의로 실패를 유도하고, 시스템의 정상적 복귀 여부를 테스트하는 기법이다.

### (2) 안전 테스트(Security Testing) : 불법적인 소프트웨어가 접근하여 시스템을 파괴하지 못하도록 소스 코드 내의 보안적인 결함을 미리 점검하는 테스트 기법이다.

### (3) 성능 테스트(Performance Testing) : 사용자의 이벤트에 시스템이 응답하는 시간, 특정 시간 내에 처리하는 업무량, 사용자 요구에 시스템이 반응하는 속도 등을 측정하는 테스트 기법이다.

### (4) 구조 테스트(Structure Testing) : 시스템의 내부 논리 경로, 소스 코드의 복잡도를 평가하는 테스트 기법이다.

### (5) 회귀 테스트(Regression Testing) : 오류를 제거하거나 수정한 시스템에서 오류 제거와 수정에 의해 새로이 유입된 오류가 없는지 확인하는 일종의 반복 테스트 기법이다.

### (6) 병행 테스트(Parallel Testing) : 변경된 시스템과 기존 시스템에 동일한 데이터를 입력 후 결과를 비교하는 테스트 기법이다.

## 5) 성능 테스트의 상세 유형

- (1) 부하 테스트(Load Testing) : 시스템에 부하를 계속 증가시키면서 시스템의 임계점을 찾는 테스트이다. 부하 테스트를 통해 병목 지점을 찾아서 병목 현상을 제거하는 과정을 반복한다.
- (2) 강도 테스트(Stress Testing) : 시스템 처리 능력 이상의 부하, 즉 임계점 이상의 부하를 가하여 비정상적인 상황에서 시스템의 동작 상태를 확인하는 테스트이다.
- (3) 스파이크 테스트(Spike Testing) : 짧은 시간에 사용자가 몰릴 때 시스템의 반응 측정 테스트이다.
- (4) 내구성 테스트(Endurance Testing) : 오랜 시간 동안 시스템에 높은 부하를 가하여 시스템 반응 테스트이다.

## 6) 정적 테스트의 리뷰 유형

- (1) 동료 검토(Peer Review) : 2~3명이 진행하는 리뷰의 형태로 요구사항 명세서 작성자가 요구사항 명세서를 설명하고, 이해관계자들이 설명을 들으면서 결함을 발견하는 형태로 진행되는 검토 기법이다.
- (2) 인스펙션(Inspection) : 소프트웨어 요구, 설계, 원시 코드 등의 저작자 외의 다른 전문가 또는 팀이 검사하여 문제를 식별하고 문제에 대한 올바른 해결을 찾아내는 형식적인 검토 기법이다.
- (3) 워크 스루(Walk Throughs) : 검토 자료를 회의 전에 배포해서 사전 검토한 후 짧은 시간 동안 회의를 진행하는 형태로 리뷰를 통해 문제 식별, 대안 조사, 개선 활동, 학습 기회를 제공하는 가장 비형식적인 검토 기법이다.

## 7) 테스트 오라클(Test Oracle) - 참샘플일

테스트의 결과가 참인지 거짓인지를 판단하기 위해서 사전에 정의된 참값을 입력하여 비교하는 기법이다.

- (1) 참(Ture) 오라클 : 모든 입력값에 대하여 기대하는 결과를 생성함으로써 발생된 오류를 모두 검출할 수 있는 오라클이다.
- (2) 샘플링(Sampling) 오라클 : 특정한 몇 개의 입력값에 대해서만 기대하는 결과를 제공해 주는 오라클이다.
- (3) 휴리스틱(Heuristic) 오라클 : 샘플링 오라클을 개선한 오라클로, 특정 입력값에 대해 올바른 결과를 제공하고, 나머지 값들에 대해서는 휴리스틱(추정)으로 처리하는 오라클이다.
- (4) 일관성 검사(Consistent) 오라클 : 애플리케이션 변경이 있을 때, 수행 전과 후의 결과값이 동일한지 확인하는 오라클이다.

## 8) 테스트 레벨 종류

- (1) 단위 테스트 : 사용자 요구사항에 대한 단위 모듈, 서브루틴 등을 테스트하는 단계이다.
- (2) 통합 테스트 : 단위 테스트를 통과한 모듈 사이의 인터페이스, 통합된 컴포넌트 간의 상호 작용을 검증하는 테스트 단계이다.
- (3) 시스템 테스트 : 통합된 단위 시스템의 기능이 시스템에서 정상적으로 수행되는지를 검증하는 테스트 단계이다.
- (4) 인수 테스트 : 계약상의 요구사항이 만족되었는지 확인하기 위한 테스트 단계이다.
  - 알파 테스트 : 선택된 사용자(회사 내의 다른 사용자 또는 실제 사용자)가 개발자 환경에서 통제된 상태로 개발자와 함께 수행하는 인수 테스트이다.
  - 베타 테스트 : 실제 환경에서 일정 수의 사용자에게 대상 소프트웨어를 사용하게 하고 피드백을 받는 인수 테스트이다.

## 9) 개별 테스트 케이스 필요 항목

- (1) 테스트 ID : 테스트 케이스를 고유하게 식별하기 위한 ID를 작성한다.
- (2) 테스트 목적 : 테스트 시 고려해야 할 중점 사항이나 테스트 케이스의 목적을 작성한다.
- (3) 테스트할 기능 : 애플리케이션의 테스트할 기능을 간략하게 작성한다.
- (4) 테스트 데이터(=입력 데이터) : 테스트 실행 시 입력할 데이터(입력값, 선택 버튼, 체크리스트 값 등)를 작성한다.
- (5) 예상 결과(=기대 결과) : 테스트 실행 후 기대되는 결과 데이터(출력 데이터, 결과 화면, 기대 동작 등)를 작성한다.
- (6) 테스트 환경 : 테스트 시 사용할 물리적, 논리적 테스트 환경, 사용할 데이터, 결과 기록 서버 등의 내용을 작성한다.
- (7) 테스트 조건(=전제 조건) : 테스트 간의 종속성, 테스트 수행 전 실행되어야 할 고려 사항 등을 작성한다.
- (8) 성공/실패 기준 : 테스트를 거친 애플리케이션 기능의 성공과 실패를 판단하는 조건을 명확하게 작성한다.
- (9) 기타 요소 : 사용자의 테스트 요구사항 중 특별히 고려해야 할 내용을 간략하게 기술한다.

## 10) 소스 코드 품질분석 도구 유형

- (1) 정적 분석 도구 : 작성된 소스 코드를 실행시키지 않고, 코드 자체만으로 코딩 표준 준수 여부, 코딩 스타일 적정 여부, 잔존 결함 발견 여부를 확인하는 코드 분석 도구이다.
- (2) 동적 분석 도구 : 애플리케이션을 실행하여 코드에 존재하는 메모리 누수 현황을 발견하고, 발생한 스레드의 결함 등을 분석하기 위한 도구이다.

## 11) 테스트 하네스(Test Harness) - 드 스슈 케이스목

애플리케이션 컴포넌트 및 모듈을 테스트하는 환경의 일부분으로, 테스트를 지원하기 위한 코드와 데이터를 말하며, 단위 또는 모듈 테스트에 사용하기 위해 코드 개발자가 작성한다.

- (1) **테스트 드라이버(Test Driver)** : 테스트 대상 하위 모듈을 호출하고, 파라미터를 전달하고, 모듈 테스트 수행 후의 결과를 도출하는 등 상향식 테스트에 필요하다.
- (2) **테스트 스텝(Test Stub)** : 제어 모듈이 호출하는 타 모듈의 기능을 단순히 수행하는 도구로 하향식 테스트에 필요하다.
- (3) **테스트 스위트(Test Suites)** : 테스트 대상 컴포넌트나 모듈, 시스템에 사용되는 테스트 케이스의 집합이다.
- (4) **테스트 케이스(Test Case)** : 입력값, 실행 조건, 기대 결과 등의 집합이다.
- (5) **테스트 시나리오(Test Scenario)** : 애플리케이션의 테스트 되어야 할 기능 및 특징, 테스트가 필요한 상황을 작성한 문서이다. 하나의 단일 테스트 시나리오가 하나 또는 여러 개의 테스트 케이스들을 포함할 수 있다.
- (6) **테스트 스크립트(Test Script)** : 자동화된 테스트 실행 절차에 대한 명세이다.
- (7) **목 오브젝트(Mock Object)** : 사용자의 행위를 조건부로 사전에 입력해 두면, 그 상황에 예정된 행위를 수행하는 객체이다.

## 12) 애플리케이션 성능 측정 지표 - 측정경자

- (1) **처리량(Throughput)** : 애플리케이션이 주어진 시간에 처리할 수 있는 트랜잭션의 수이다.
- (2) **응답 시간(Response Time)** : 사용자 입력이 끝난 후, 애플리케이션의 응답 출력이 개시될 때까지의 시간이다. (애플리케이션의 경우 메뉴 클릭 시 해당 메뉴가 나타나기까지 걸리는 시간을 의미)
- (3) **경과 시간(Turnaround Time)** : 애플리케이션에 사용자가 요구를 입력한 시점부터 트랜잭션을 처리 후 그 결과의 출력이 완료할 때까지 걸리는 시간이다.
- (4) **자원 사용률(Resource Usage)** : 애플리케이션이 트랜잭션을 처리하는 동안 사용하는 CPU 사용량, 메모리 사용량, 네트워크 사용량을 의미한다.

## 13) 리팩토링(Refactoring)

유지보수 생산성 향상을 목적으로 기능을 변경하지 않고, 복잡한 소스 코드를 수정, 보완하여 가용성 및 가독성을 높이는 기법이다.

[리팩토링의 목적]

- (1) **유지보수성 향상** : 복잡한 코드의 단순화, 소스의 가독성 향상
- (2) **유연한 시스템** : 소프트웨어 요구사항 변경에 유연한 대응
- (3) **생산성 향상** : 정제 및 최적화된 소스의 재사용
- (4) **품질향상** : 소프트웨어 오류발견이 용이하여 품질향상

## 11단원 - 응용 SW 기초 기술 활용

### 1) 메모리 관리 기법

- (1) **반입 기법** : 주기억장치에 적재할 다음 프로세스의 반입 시기를 결정하는 기법이다. (메모리로 적재 시기 결정 - when)
- (2) **배치 기법** : 디스크에 있는 프로세스를 주기억장치의 어느 위치에 저장할 것인지 결정하는 기법이다. (메모리 적재 위치 결정 - where)
  - **최초 적합(First Fit)** : 프로세스가 적재될 수 있는 가용 공간 중에서 첫 번째 분할에 할당하는 방식이다.
  - **최적 적합(Best Fit)** : 가용 공간 중에서 가장 크기가 비슷한 공간을 선택하여 프로세스를 적재하는 방식이다. (공백 최소화 장점이 있음)
  - **최악 적합(Worst-Fit)** : 프로세스의 가용 공간 중에서 가장 큰 공간에 할당하는 방식이다.
- (3) **할당 기법** : 실행해야 할 프로세스를 주기억장치에 어떤 방법으로 할당할 것인지 결정하는 기법이다. (메모리 적재 방법 결정 - how)
  - **페이징 기법(Paging)** : 가상기억장치 내의 프로세스를 일정하게 분할하여 주기억장치의 분산된 공간에 적재시킨 후 프로세스를 수행시키는 기법이다.
  - **세그멘테이션 기법(Segmentation)** : 가상기억장치 내의 프로세스를 가변적인 크기의 블록으로 나누고 메모리를 할당하는 기법이다.

(4) **교체 기법** : 재배치 기법으로 주기억장치에 있는 프로세스 중 어떤 프로세스를 제거할 것인지를 결정하는 기법이다. (메모리 교체 대상 결정 - who)

- **FIFO(First In First Out)** : 각 페이지가 주기억장치에 적재될 때마다 그때의 시간을 기억시켜 가장 먼저 들어와 가장 오래 있던 페이지를 교체하는 기법이다. → 선입선출
- **LRU(Least Recently Used)** : 사용된 시간을 확인하여 가장 오랫동안 사용되지 않은 페이지를 선택하여 교체하는 기법이다.
- **LFU(Least Frequently Used)** : 사용된 횟수를 확인하여 참조 횟수가 가장 적은 페이지를 선택하여 교체하는 기법이다.
- **OPT(OPTimal Replacement)** : 앞으로 가장 오랫동안 사용하지 않을 페이지를 교체하는 기법이다.
- **NUR(Not Used Recently)** : 최근에 사용하지 않은 페이지를 교체하는 기법이다. 최근에 사용되지 않은 페이지는 앞으로도 사용되지 않을 가능성이 크다는 전제로, LRU에서 나타나는 시간적인 오버헤드를 줄일 수 있다.
- **SCR(Second Chance Replacement)** : 가장 오랫동안 주기억장치에 있던 페이지 중 자주 사용되는 페이지의 교체를 방지하기 위한 기법으로 FIFO 기법의 단점을 보완하는 기법이다.

2) **선점형 스케줄링(Preemptive Scheduling)** : 하나의 프로세스가 CPU를 차지하고 있을 때, 우선순위가 높은 다른 프로세스가 현재 프로세스를 중단시키고 CPU를 점유하는 스케줄링 방식이다.

- (1) **라운드 로빈(RR ; Round Robin)** : 프로세스는 같은 크기의 CPU 시간을 할당, 프로세스가 할당된 시간 내에 처리 완료를 못하면 준비 큐 리스트의 가장 뒤로 보내지고, CPU는 대기 중인 다음 프로세스로 넘어가는 스케줄링 기법
- (2) **SRT(Shortest Remaining Time First)** : 가장 짧은 시간이 소요되는 프로세스를 먼저 수행, 남은 처리시간이 더 짧다고 판단되는 프로세스가 준비 큐에 생기면 언제라도 프로세스가 선점되는 스케줄링 기법으로 비선점 방식의 스케줄링 기법에 선점 방식을 도입한 기법
- (3) **다단계 큐(MLQ ; Multi Level Queue)** : 작업들을 여러 종류 그룹으로 분할, 여러 개의 큐를 이용하여 상위단계 작업에 의한 하위단계 작업이 선점 당하는 스케줄링 기법
- (4) **다단계 피드백 큐(MLFQ ; Multi Level Feedback Queue)** : FCFS(FIFO)와 라운드 로빈 스케줄링 기법을 혼합한 것으로, 새로운 프로세스는 높은 우선순위, 프로세스의 실행시간이 길어질수록 점점 낮은 우선순위 큐로 이동하고 마지막 단계는 라운드 로빈 방식을 적용하는 스케줄링 기법

3) **비선점형 스케줄링(Non Preemptive Scheduling)**

한 프로세스가 CPU를 할당받으면 작업 종료 후 CPU 반환 시까지 다른 프로세스는 CPU 점유가 불가능한 스케줄링 방식이다.

- (1) **우선순위(Priority)** : 프로세스별로 우선순위가 주어지고, 우선 순위에 따라 CPU를 할당하는 스케줄링 기법이다. (동일 순위는 FCFS)
- (2) **기한부(Deadline)** : 작업들이 명시된 시간이나 기한 내에 완료되도록 계획하는 스케줄링 기법
- (3) **FCFS(First Come First Service)** : 프로세스가 대기 큐에 도착한 순서에 따라 CPU를 할당하는 스케줄링 기법 (FIFO 알고리즘이라고도 함)
- (4) **SJF(Shortest Job First)** : 프로세스가 도착하는 시점에 따라 그 당시 가장 작은 서비스 시간을 갖는 프로세스가 종료 시까지 자원을 점유하는 스케줄링 기법
- (5) **HRN(Highest Response Ration Next)** : 대기 중인 프로세스 중 현재 응답률이 가장 높은 것을 선택하는 스케줄링 기법  
→ HRN의 우선순위 = (대기 시간 + 서비스 시간) / 서비스 시간

4) **교착상태(Deadlock)** : 다중 프로세싱 환경에서 두 개 이상의 프로세스가 특정 자원할당을 무한정 대기하는 상태이다.

(1) **교착상태 발생 조건 - 상점비환**

- **상호 배제(Mutual Exclusive)** : 프로세스가 자원을 배타적으로 점유하여 다른 프로세스가 그 자원을 사용할 수 없는 상태이다.
- **점유와 대기(Hold & Wait)** : 한 프로세스가 자원을 점유하고 있으면서 또 다른 자원을 요청하여 대기하고 있는 상태이다.
- **비선점(Non Preemption)** : 한 프로세스가 점유한 자원에 대해 다른 프로세스가 선점할 수 없고, 오직 점유한 프로세스만이 해제 가능한 상태이다.
- **환형 대기(Circular Wait)** : 두 개 이상의 프로세스 간 자원의 점유와 대기가 하나의 원형을 구성한 상태이다.

(2) **교착상태 해결 방법 - 예회발복**

- **예방(Prevention)** : 상호 배제를 제외한 나머지 교착상태 발생 조건을 위배하는 방안
- **회피(Avoidance)** : 안전한 상태를 유지할 수 있는 요구만 수락(프로세스별 자원 최대요구량 확보)
- **발견(Detection)** : 시스템의 상태를 감시 알고리즘 통해 교착 상태 검사
- **복구(Recovery)** : 교착상태가 없어질 때까지 프로세스를 순차적으로 Kill하여 제거, 희생자 선택해야 하고 기아 상태 발생

## 5) 디스크 스케줄링

사용할 데이터가 디스크상의 여러 곳에 저장되어 있을 경우, 데이터를 액세스하기 위해 디스크 헤드를 움직이는 경로를 결정하는 기법이다.

- (1) FCFS(First Come First Served) = FIFO : 디스크 대기 큐에 가장 먼저 들어온 트랙에 대한 요청을 먼저 서비스하는 기법이다.
- (2) SSTF(Shortest Seek Time First) : 현재 위치에서 탐색거리가 가장 짧은 트랙에 대한 요청을 먼저 서비스하는 기법이다.
- (3) SCAN : 현재 헤드의 위치에서 진행 방향이 결정되면 탐색 거리가 짧은 순서에 따라 그 방향의 모든 요청을 서비스하고, 끝까지 이동한 후 역방향의 요청 사항을 서비스하는 기법이다.
- (4) C-SCAN(Circular SCAN) : 항상 바깥쪽에서 안쪽으로 움직이며 가장 짧은 탐색 거리를 갖는 요청을 서비스하는 기법이다. 안쪽 끝까지 이동했으면 다시 바깥쪽부터 탐색하는 방법으로 비교적 공평한 기법이다.
- (5) LOOK(=엘리베이터 알고리즘) : SCAN을 기초로 사용하는 기법으로 진행 방향으로 더 이상의 요청이 없으면 역방향으로 진행하는 기법이다.
- (6) N-STEP SCAN : SCAN 기법을 기초로 하며 어떤 방향의 진행이 시작될 당시에 대기 중이던 요청들만 서비스하고, 진행 도중 도착한 요청들은 한꺼번에 모아서 다음의 반대 진행 방향으로 진행할 때 서비스하는 기법이다.
- (7) SLTF(Shortest Latency Time First) : 섹터 큐잉(Sector Queuing)이라고 하며, 회전지연시간 최적화를 위해 구현된 기법이다. 디스크 헤드가 특정 실린더에 도착하며 그 실린더 내의 여러 트랙에 대한 요청들을 검사한 후, 회전지연시간이 가장 짧은 요청부터 서비스하는 기법이다.

## 6) OSI 7계층 - 아파서 티내다, 피나다

▼ OSI 7 Layer의 특징

| 계층 이름                         | 설명                                                                       | 프로토콜               | 전송단위              | 장비            |
|-------------------------------|--------------------------------------------------------------------------|--------------------|-------------------|---------------|
| 응용 계층<br>(Application Layer)  | • 사용자와 네트워크 간 응용서비스 연결, 데이터 생성                                           | • HTTP<br>• FTP    | 데이터<br>(Data)     | 호스트<br>(PC 등) |
| 표현 계층<br>(Presentation Layer) | • 데이터 형식 설정, 부호교환, 암호·복호화, 압축                                            | • JPEG<br>• MPEG   |                   |               |
| 세션 계층<br>(Session Layer)      | • 송수신 간의 논리적인 연결<br>• 연결 접속, 동기제어                                        | • RPC<br>• NetBIOS |                   |               |
| 전송 계층<br>(Transport Layer)    | • 송수신 프로세스 간의 연결<br>• 신뢰성 있는 통신 보장<br>• 데이터 분할, 재조립, 흐름 제어, 오류 제어, 혼잡 제어 | • TCP<br>• UDP     | 세그먼트<br>(Segment) | L4<br>스위치     |
| 네트워크 계층<br>(Network Layer)    | • 단말기 간 데이터 전송을 위한 최적화된 경로 제공                                            | • IP<br>• ICMP     | 패킷<br>(Packet)    | 라우터           |
| 데이터링크 계층<br>(Data Link Layer) | • 인접 시스템 간 데이터 전송, 전송 오류 제어<br>• 동기화, 오류 제어, 흐름 제어, 회선 제어                | • HDLC<br>• PPP    | 프레임<br>(Frame)    | 브리지,<br>스위치   |
| 물리 계층<br>(Physical Layer)     | • 0과 1의 비트 정보를 회선에 보내기 위한 전기적 신호 변환                                      | • RS-232C          | 비트<br>(Bit)       | 허브,<br>리피터    |

## 7) 프로토콜(Protocol) : 서로 다른 시스템이나 기기들 간의 데이터 교환을 원활히 하기 위한 표준화된 통신규약이다.

- 구문(Syntax) : 시스템 간의 정보 전송을 위한 데이터 형식, 코딩, 신호 레벨 등의 규정
- 의미(Semantic) : 시스템 간의 정보 전송을 위한 제어 정보로 조정과 여러 처리를 위한 규정
- 타이밍(Timing) : 시스템 간의 정보 전송을 위한 속도 조절과 순서 관리 규정

## 8) 네트워크 계층 프로토콜

- (1) IP(Internet Protocol) : 송수신 간의 패킷 단위로 데이터를 교환하는 네트워크에서 정보를 주고받는 데 사용하는 통신 프로토콜이다.
- (2) ARP(Address Resolution Protocol) : IP 네트워크 상에서 IP 주소를 MAC 주소(물리 주소)로 변환하는 프로토콜이다.
- (3) RARP(Reverse Address Resolution Protocol) : IP 호스트가 자신의 물리 네트워크 주소(MAC)는 알지만 IP 주소를 모르는 경우, 서버로부터 IP 주소를 요청하기 위해 사용하는 프로토콜이다.

(4) ICMP(Internet Control Message Protocol) : IP 동작 과정에서 전송 오류가 발생하는 경우에 오류 정보를 전송하는 목적으로 사용하는 프로토콜이다.

(5) IGMP(Internet Group Management Protocol) : 인터넷 그룹 관리 프로토콜은 호스트 컴퓨터와 인접 라우터가 멀티캐스트 그룹 멤버십을 구성하는 데 사용하는 통신 프로토콜이다.

(6) 라우팅 프로토콜(Routing Protocol) : 데이터 전송을 위해 목적지까지 갈 수 있는 여러 경로 중 최적의 경로를 설정해주는 라우터 간의 상호 통신 프로토콜이다.

- IPv4 : 8bit씩 4부분으로 나뉜 10진수
- IPv6 : 16bit씩 8부분으로 나뉜 16진수

9) 내부 라우팅 프로토콜(IGP ; Interior Gateway Protocol) : 동일한 AS 내의 라우팅에 사용되는 프로토콜이다.

(1) RIP(Routing Information Protocol) : AS(Autonomous System ; 자치 시스템, 자율 시스템) 내에서 사용하는 거리 벡터 알고리즘에 기초하여 개발된 내부 라우팅 프로토콜이다.

→ 특징 : 벨만-포드 알고리즘 사용, 15홉 제한, UDP 사용, 30초마다 정보 공유

(2) OSPF(Open Shortest Path First) : 규모가 크고 복잡한 TCP/IP 네트워크에서 RIP의 단점을 개선하기 위해 자신을 기준으로 링크 상태 알고리즘을 적용하여 최단 경로를 찾는 라우팅 프로토콜이다.

→ 특징 : 다익스트라 알고리즘 사용, 라우팅 매트릭 지정, AS 분할 사용, 홉 카운트 무제한, 멀티캐스팅 지원

10) 외부 라우팅 프로토콜(EGP ; Exterior Gateway Protocol)

서로 다른 AS 간 라우팅 프로토콜로 게이트웨이 간의 라우팅에 사용되는 프로토콜이다.

(1) BGP(Border Gateway Protocol)

AS 상호 간에 경로 정보를 교환하기 위한 라우팅 프로토콜이다.

11) 라우팅 알고리즘

데이터는 송신 측으로부터 수신 측까지 데이터를 전달하는 과정에서 다양한 물리적인 장치들을 거쳐간다. 목적지까지의 최적 경로를 산출하기 위한 법칙이 라우팅 알고리즘이다.

(1) 거리 벡터 알고리즘(Distance Vector Algorithm) : 인접 라우터와 정보를 공유하여 목적지까지의 거리와 방향을 결정하는 라우팅 프로토콜 알고리즘이다. → 벨만-포드(Bellman-Ford) 알고리즘 사용

(2) 링크 상태 알고리즘(Link State Algorithm) : 링크 상태 정보를 모든 라우터에 전달하여 최단 경로 트리를 구성하는 라우팅 프로토콜 알고리즘이다. → 다익스트라(Dijkstra) 알고리즘 사용

12) 패킷 교환 방식의 종류

(1) 데이터그램(Datagram) 방식 : 연결 경로를 확립하지 않고 각각의 패킷을 순서에 무관하게 독립적으로 전송하는 방식이다.

(2) 가상 회선(Virtual Circuit) 방식 : 패킷이 전송되기 전에 송수신 스테이션 간의 논리적인 통신 경로를 미리 설정하는 방식이다.